



UNIVERSITÉ DE
MONTPELLIER



Rapport d'Audit Technique

DELLAROLI Romain

DUBOIS Clément

LOPEZ Julian

LUBEN Bastien

Table des matières :

I. Qualité du Code et Architecture.....	3
1.1 Violation du principe DRY.....	3
1.2 Violation des principes SOLID.....	3
1.3 Logique Métier dans les Vues (Violation SRP/MVC).....	4
1.4 Problèmes de Testabilité.....	5
1.5 Code Mort et Documentation.....	5
1.6 Mauvaise Gestion des Erreurs et de la Validation.....	5
1.7 Problèmes de Performance.....	6
II. Audit de Sécurité.....	7
2.1 Injection SQL.....	7
2.2 Cross-Site Scripting (XSS).....	8
2.3 Hachage de Mots de Passe Défaillant.....	8
2.4 Stockage de Mots de Passe en Clair.....	8
2.5 Fuite de Données (Cookies).....	9
2.6 Email demandé mais jamais vérifié.....	9
2.7 Absence de protection CSRF (Cross-Site Request Forgery).....	10
2.8 Contrôle d'Accès Défaillant.....	10
2.9 Incohérence des Validations (Front vs Back).....	11
III. Modélisation de la Base de Données.....	12
3.1 Analyse critique de l'existant.....	12
3.2 Proposition de nouveau schéma ("Sain").....	13

Rapport d'Audit Technique - Application "Noodle"

Ce document présente l'analyse technique de l'application Noodle. L'audit révèle de graves déficiences en matière de sécurité, de conception architecturale et de modélisation des données, rendant l'application impropre à la mise en production en l'état.

I. Qualité du Code et Architecture

1.1 Violation du principe DRY

La structure de la base de données force une duplication massive de données. Les informations de l'utilisateur (nom, prénom, password) sont répétées pour chaque créneau de chaque sondage. Cela mène à une redondance, voire une incohérence des données stockées (les prénoms qui se désynchronisent par exemple).

Le code de connexion à la base de données et les requêtes SQL sont dispersés et parfois dupliqués (ex: logique de récupération similaire dans `SondageRepository` et `UtilisateurRepository`).

HTML et Vues :

Les fichiers `src/vue/sondage/formulaireCreationSondage.php` et `src/vue/sondage/formulaireMiseAJourSondage.php` contiennent 90% de code identique. Les champs (titre, durée, etc.) sont dupliqués.

Solution : Extraire les champs communs dans un fichier partiel `champsFormulaireSondage.php` inclus par les deux vues.

1.2 Violation des principes SOLID

Single Responsibility Principle (SRP) :

Les classes "Contrôleur" (ex: `ControleurSondage`) sont surchargées. Elles gèrent la validation des entrées, la logique métier, l'interaction avec le repository et le choix de la vue.

Problème de l'Open/Closed Principle (OCP) & Dependency Inversion (DIP) :

Les contrôleursinstancient directement les dépôts avec `new SondageRepository()`. Le code

est fortement couplé à l'implémentation spécifique (SQL). Impossible d'étendre le système pour utiliser une autre source de données (ex: API externe, fichiers, mock pour tests) sans modifier tous les contrôleurs.

Solution : Utiliser l'Injection de Dépendance ou une Factory pour fournir les instances de Repository.

MessageFlash::lireTousMessages() n'est pas Open/Closed :

Les types de messages sont hardcodés dans un tableau ["success", "info", "warning", "danger"]. Si on veut ajouter un nouveau type (ex: "notice"), il faut modifier le code source de la méthode.

Solution : Lire dynamiquement toutes les clés présentes en session, ou utiliser un enum PHP 8.1 pour définir les types autorisés de manière extensible.

Gestion des Disponibilités non Open/Closed :

Le type de disponibilité ("dispo", "indispo", etc.) est géré via des chaînes de caractères comparées manuellement (ex: \$typeDisponibilite == "dispo" dans la vue). Aucune structure ne contraint les valeurs possibles.

Solution : Utiliser un enum PHP 8.1 (enum TypeDisponibilite: string { case Disponible = 'dispo'; case PeutEtre = 'peutetre'; ... }) pour typer et contraindre les valeurs.

1.3 Logique Métier dans les Vues (Violation SRP/MVC)

L'application actuelle rencontre de nombreuses erreurs de logique métier.

Vous trouverez ci-dessous une liste de tous nos relevés :

Fichier : src/vue/sondage/sondage.php

Calculs effectués dans la vue (qui devraient être dans le contrôleur) :

Ligne 80 : Calcul de la date de fin du créneau \$dateFin = (new DateTime(...))->modify(...). C'est de la logique métier.

Ligne 82 : Comparaison pour déterminer si le créneau courant est le meilleur créneau (\$creneau->getDateEtHoraire()->format(...) === \$meilleurCreneau->...). C'est de la logique de présentation conditionnelle basée sur un calcul métier.

Ligne 105 : Construction des initiales des participants (\$disponibilite->getParticipant()->getPrenom()[0].\$disponibilite->getParticipant()->getNom()[0]).

Solution : Pré-calculer ces valeurs dans le contrôleur et les passer à la vue en tant que variables simples.

1.4 Problèmes de Testabilité

Les Repositories sont hardcodés (instanciés directement avec `new`) dans les contrôleurs. Il est impossible de tester un contrôleur sans déclencher de vrais accès à la base de données (effets de bord). Le code est intimement non-testable unitairement.

Solution : Utiliser l'injection de dépendance avec des interfaces (`RepositoryInterface`), permettant de substituer un `MockRepository` lors des tests.

1.5 Code Mort et Documentation

Nous avons détecté une documentation non précise ou absente, en plus de bout de code n'ayant aucune utilité dans l'application. Vous trouverez des exemples ci-dessous.

Méthode `tel()` inutile (`ConnexionUtilisateur.php`, L45-49) : Retourne simplement 5 avec un commentaire `//TO-DO`. Code mort appelé dans `ControleurUtilisateur::mettreAJour()` (L135 : `$x = ConnexionUtilisateur::tel();`) sans que `$x` ne soit jamais utilisé.

Méthode `important()` vide (`MotDePasse.php`, L31-40) : Méthode sans contenu, avec uniquement des commentaires informels entre développeurs ("Je crois que ça ne marche pas hahahaha"). C'est du code mort avec des commentaires non-professionnels qui n'apportent rien. (Remarque : devenez humoriste monsieur 😊)

Méthode `unlog()` non implémentée (`SondageRepository.php`, L155-160) : Affiche simplement "To-DO unlog". Responsabilité mal placée (un Repository ne devrait pas gérer la déconnexion).

Documentation absente : Aucune PHPDoc significative sur les méthodes publiques, aucun README technique, aucun commentaire expliquant la logique métier complexe.

1.6 Mauvaise Gestion des Erreurs et de la Validation

Aucun appel à la base de données n'est encapsulé dans un bloc `try/catch`. Si une requête SQL échoue (connexion perdue, erreur de syntaxe, contrainte violée), l'exception PDO remonte sans être interceptée, provoquant un crash brut avec potentiellement des informations techniques sensibles (nom de table, requête SQL) affichées à l'utilisateur. Chaque interaction avec la BDD devrait être protégée par un `try/catch` avec une gestion d'erreur propre (log côté serveur + message générique côté utilisateur).

1.7 Problèmes de Performance

Base de données non normalisée : La table unique `app_db` contient toutes les données. Pour récupérer un sondage, le système doit lire des lignes contenant énormément de données redondantes (infos utilisateur, etc.).

Requêtes SQL inefficaces :

`SondageRepository::recupererPar` fait un `SELECT *` sur une table qui contient tout l'historique. Le volume de données transféré est inutilement grand.

L'utilisation de `JSON_CONTAINS` sur une colonne texte pour filtrer les invitations est extrêmement coûteuse (Full Table Scan) car elle ne peut pas bénéficier d'indexation standard efficace sur des bases MySQL anciennes ou mal configurées.

Logique PHP gourmande : Le traitement des données JSON (invitations, disponibilités) se fait en PHP après récupération de la base, ce qui consomme mémoire et CPU inutilement alors qu'une base relationnelle bien faite gèrerait cela nativement.

Pas de dynamisme côté client : Actuellement, toutes les actions de l'utilisateur se traduisent par une requête HTTP complète vers le serveur, traitée par le contrôleur PHP, ce qui implique un rechargement de page et une nouvelle exécution du cycle serveur à chaque interaction. Cette architecture entraîne une surcharge inutile du serveur et peut poser des problèmes de performance, notamment en cas d'augmentation du nombre d'utilisateurs. Une amélioration consisterait à dynamiser certaines interactions côté client à l'aide de JavaScript (par exemple via des requêtes AJAX/fetch), afin de limiter les rechargements complets de page et de réduire le volume de traitements serveur, tout en améliorant la fluidité de l'expérience utilisateur.

II. Audit de Sécurité

L'application contient de multiples failles critiques exploitables trivialement.

2.1 Injection SQL

Problème : Les requêtes SQL dans les répertoires (Repositories) concatènent directement des variables utilisateurs sans utiliser de requêtes préparées systématiquement.

Faillle 1 : Récupération de sondage

Fichier : src/Modele/Repository/SondageRepository.php

Code : `SELECT * FROM app_db WHERE $critere = '$valeur' ...` (méthode `recupererPar`)

Scénario d'exploitation : Un attaquant appelle l'URL pour voir un sondage caché : `...&codeSondage=' OR '1'='1`.

Résultat : Le code SQL devient `SELECT ... WHERE codeSecretSondage = " OR '1'='1'`, retournant tous les sondages de la base, contournant la protection par code secret.

Faillle 2 : Récupération de compte par email

Fichier : src/Modele/Repository/UtilisateurRepository.php

Code : `SELECT ... WHERE emailProprietaire = '$email'` (méthode `recupererParEmail`)

Scénario d'exploitation : Dans le formulaire de récupération, saisir `' OR '1'='1`.

Résultat : La requête retourne tous les utilisateurs de la base (avec leur mot de passe en clair).

De manière générale, la combinaison des failles d'injection SQL et du contrôle d'accès défaillant rend la base de données entièrement compromise. Un attaquant capable d'injecter du code SQL arbitraire peut non seulement contourner les protections existantes, mais également exécuter des requêtes destructrices ou d'exfiltration de données. Concrètement, cela signifie qu'il est possible de lire, modifier ou supprimer n'importe quelle information stockée, voire d'exécuter des commandes critiques comme `DROP TABLE` ou `DROP DATABASE` si les privilèges du compte SQL le permettent. En parallèle, l'absence de vérification stricte des droits côté serveur permet d'agir sur des ressources appartenant à d'autres utilisateurs simplement en manipulant les paramètres d'URL. L'association de ces deux vulnérabilités place l'ensemble du système dans une situation de compromission totale, où un attaquant peut agir sur la base de données comme bon lui semble.

Correctif : Utiliser systématiquement des requêtes préparées (`pdo->prepare()`) avec des marqueurs nommés (ex: `:email`) pour toutes les variables, comme c'est déjà fait partiellement ailleurs dans le code.

2.2 Cross-Site Scripting (XSS)

Problème : Les données utilisateurs sont affichées dans les vues sans échappement (htmlspecialchars) constant.

Faible : Titre du sondage non échappé.

Fichier : src/vue/sondage/sondage.php

Code : `<span><?=$sondage->getTitre()?></span>` (Ligne 23).

Scénario d'exploitation : Un attaquant crée un sondage avec le titre `<script>alert(document.cookie)</script>`.

Résultat : Lorsque la victime visite la page du sondage, le script s'exécute, permettant potentiellement le vol de session.

Correctif : Passer systématiquement toutes les données affichées dans la fonction htmlspecialchars().

2.3 Hachage de Mots de Passe Défaillant

Problème Critique : La classe MotDePasse (src/Lib/MotDePasse.php) implémente un hachage cryptographiquement faible.

Pas de sel (salt) : La fonction hacher() utilise hash('sha256', \$mdpClair) sans sel. Deux utilisateurs ayant le même mot de passe auront le même hash, facilitant les attaques par tables arc-en-ciel.

Poivre mal appliqué : Le poivre (\$poivre) est simplement concaténé devant le hash (return \$poivre . hash(...)) au lieu d'être intégré au processus de hachage. Il est donc trivialement séparable du hash.

Conséquence : Si deux utilisateur ont le même mot de passe, ils auront le même mot de passe haché stocké dans la BD.

Correctif : Utiliser password_hash() (bcrypt/argon2) natif de PHP, qui gère automatiquement le sel et utilise un algorithme lent par conception.

2.4 Stockage de Mots de Passe en Clair

Problème Critique : Les mots de passe sont stockés en clair dans la base de données (colonne mdpProprietaire) en plus d'une version hachée.

Preuve : SondageRepository.php ligne 48 hydrate l'objet Utilisateur avec le mot de passe en clair.

Aggravation : La fonctionnalité "Récupérer mon compte" (resultatRecuperationCompte.php) affiche les mots de passe en clair de tous les comptes associés à l'email (ou de tous les comptes si exploité via SQLi).

Correctif : Supprimer la colonne mdpProprietaire. Ne stocker QUE le hash. Pour la récupération, envoyer un lien de réinitialisation par email, ne jamais renvoyer l'ancien mot de passe.

2.5 Fuite de Données (Cookies)

Problème : Les identifiants (Login et Mot de passe en clair) sont stockés dans les cookies du navigateur.

Fichier : src/Controleur/ControleurUtilisateur.php

Code : `Cookie::enregistrer("cookie_mdp", $_REQUEST["mdp"]);`

Usage : Ces cookies servent à pré-remplir le formulaire de connexion (autocomplétion "maison"). Le mot de passe en clair de l'utilisateur transite et persiste dans le navigateur.

Impact : Toute personne ayant accès au poste de l'utilisateur ou interceptant les cookies (sur HTTP non sécurisé) vole les identifiants immédiatement.

Correctif : Ne jamais stocker de mot de passe en cookie. Utiliser des sessions PHP sécurisées (session_start) avec un identifiant de session opaque. L'autocomplétion doit être déléguée au navigateur (autocomplete natif).

2.6 Email demandé mais jamais vérifié

Problème : L'application demande une adresse email à l'inscription et la stocke. Cependant, aucune vérification par envoi d'email n'est effectuée. L'email sert même de base à la récupération de compte (qui affiche le mot de passe en clair).

Impact : N'importe qui peut s'inscrire avec l'email d'un tiers, puis utiliser la récupération de compte pour récupérer les identifiants de la vraie personne si elle s'inscrit plus tard.

Correctif : Envoyer un email de confirmation avec un lien de validation unique avant d'activer le compte.

2.7 Absence de protection CSRF (Cross-Site Request Forgery)

Problème : Aucune mesure n'est prise pour vérifier que les actions (création de sondage, modification de profil) proviennent bien de l'utilisateur via le formulaire légitime et non via un lien piégé par un attaquant.

Scénario : Un attaquant fait cliquer la victime connectée sur un lien <http://site-legacy/controleurFrontal.php?action=supprimerSondage&idSondage=123>. Le sondage est supprimé immédiatement sans confirmation ni jeton de sécurité.

Correctif : Ajouter un jeton anti-CSRF (token aléatoire stocké en session) dans chaque formulaire et le vérifier côté serveur lors de la soumission.

2.8 Contrôle d'Accès Défaillant

Problème Critique : Plusieurs actions sensibles ne vérifient pas si l'utilisateur connecté est bien autorisé à agir sur la ressource ciblée. L'attaquant peut modifier ou supprimer des données n'importe qui en changeant simplement un paramètre dans l'URL.

Faible 1 : Suppression et Modification de Compte (ControleurUtilisateur)

Fichier : src/Controleur/ControleurUtilisateur.php

Méthodes : `supprimer()` et `mettreAJour()`.

Code : Le login cible est récupéré via `$_REQUEST['login']`. Aucune vérification n'est faite pour s'assurer que `$_REQUEST['login'] === ConnexionUtilisateur::getLoginUtilisateurConnecte()`.

Scénario : Un utilisateur connecté envoie une requête `?action=supprimer&login=admin`. Le compte administrateur est supprimé.

Scénario 2 : Un utilisateur change le mot de passe de n'importe qui via `?action=mettreAJour&login=victime&mdp=123`.

Faible 2 : Suppression de Créneau (ControleurCreneau)

Fichier : src/Controleur/ControleurCreneau.php

Méthode : `supprimerCreneau()`.

Code : Le code récupère le créneau via l'ID sondage et la date, puis effectue `creneauRepository->supprimer(...)`. Il n'y a aucune vérification que l'utilisateur est administrateur du sondage.

Scénario : Un utilisateur malveillant peut supprimer tous les créneaux de tous les sondages de la plateforme (en bruteforçant les IDs).

Correctif :

Pour les actions liées au compte utilisateur : Toujours utiliser l'ID de l'utilisateur stocké en session (`ConnexionUtilisateur::getLoginUtilisateurConnecte()`) et ignorer le paramètre login de l'URL pour la cible.

Pour les ressources (Sondages, Créneaux) : Vérifier systématiquement les droits (`estProprietaire()` ou `estAdministrateur()`) après avoir chargé l'objet et avant d'effectuer l'action.

2.9 Incohérence des Validations (Front vs Back)

Problème : L'application se repose quasi-exclusivement sur les validations HTML5 (attributs `required`, `min`, `maxlength`, `pattern`) pour garantir l'intégrité des données. Le serveur (PHP) ne revérifie pas ces contraintes.

Impact : Un attaquant peut contourner toutes les protections HTML (via curl, Postman ou suppression d'attributs dans le navigateur) pour envoyer des données invalides.

Écarts Identifiés :

Utilisateur : Login

Validation Front (HTML) : `minlength="3"`, `maxlength="30"`

Validation Back (PHP Actuelle) : `isset` uniquement

Manquement Critique : Pas de limite de taille. Login vide ou de 1Go possible.

Utilisateur : MDP

Validation Front (HTML) : `minlength="6"`, `maxlength="50"` + Regex Complexe (Maj+Min+Chiffre+Special)

Validation Back (PHP Actuelle) : `isset` uniquement

Manquement Critique : Mot de passe "a" ou vide accepté par le back-end. La sécurité du mot de passe est nulle.

Utilisateur : Email

Validation Front (HTML) : `type="email"`, `maxlength="255"`

Validation Back (PHP Actuelle) : `filter_var(EMAIL)`

Manquement Critique : Correct (seul point validé).

Utilisateur : Nom/Prénom.

Validation Front (HTML) : `minlength="1"`, `maxlength="30"`

Validation Back (PHP Actuelle) : `isset` uniquement

Manquement Critique : Pas de limite de taille et peut ne pas contenir uniquement des

lettres (Remarques : grâce à cette faille le 14 ème enfant d'Elon Musk peut s'inscrire sur le site).

Sondage : Titre

Validation Front (HTML) : minlength="3", maxlength="50"

Validation Back (PHP Actuelle) : isset uniquement

Manquement Critique : Titre vide ou très long accepté.

Sondage : Durée/MinPart.

Validation Front (HTML) : type="number", min="5" / min="1"

Validation Back (PHP Actuelle) : isset uniquement

Manquement Critique : Valeurs négatives ou non-numériques acceptées (si typage PHP faible).

Créneau : Date

Validation Front (HTML) : min="now + 5min"

Validation Back (PHP Actuelle) : isset uniquement

Manquement Critique : Création de créneaux dans le passé possible.

Correctif :

Dupliquer systématiquement toutes les règles de validation côté serveur.

Utiliser une librairie de validation ou une classe utilitaire Validator pour centraliser ces règles (ex: Validator::validerLongueur(\$str, 3, 30)).

Validation stricte du mot de passe côté serveur avec la même Regex que le front.

III. Modélisation de la Base de Données

3.1 Analyse critique de l'existant

La base actuelle repose sur une Table Unique (app_db), ce qui est une abomination en termes de modélisation relationnelle.

Clé Primaire Actuelle :

(loginProprietaire, idSondage, dateEtHoraire)

Critique : L'inclusion de loginProprietaire viole la 2ème et 3ème Forme Normale. Le nom, prénom, email, mot de passe dépendent uniquement de loginProprietaire, pas de idSondage ou dateEtHoraire.

De même, titreSondage dépend de idSondage, mais est répété pour chaque créneau (dateEtHoraire).

Anomalies :

Anomalie de Mise à Jour : Si l'utilisateur change de mot de passe, il faut mettre à jour des centaines de lignes (tous les créneaux de tous ses sondages).

Anomalie d'Insertion : Impossible de créer un utilisateur sans créer immédiatement un sondage et un créneau (car la PK l'exige).

Anomalie de Suppression : Si on supprime le dernier créneau du dernier sondage d'un utilisateur, on perd toutes les informations de cet utilisateur (suppression du compte accidentelle).

3.2 Proposition de nouveau schéma ("Sain")

Attributs supprimés pour la sécurité :

mdpProprietaire (mot de passe en clair) : SUPPRIMÉ.

invitations (JSON) : SUPPRIMÉ (remplacé par table de jointure).

disponibilites (JSON) : SUPPRIMÉ (remplacé par table de jointure + table gérant les différents types).

Schéma relationnelle :

UTILISATEUR(loginUtilisateur , nomUtilisateur, prenomUtilisateur, emailUtilisateur, emailAVerifier, nonce, mdpHache)

SONDAGE(idSondage , dateCreationSondage, **codeSecretSondage**, titreSondage, dureeMinutesEvenement, minimumParticipants, organiserAuPlusTot, eviterWeekend, loginProprietaire#)

Lorsqu'on crée un sondage on ajoute automatiquement (trigger) loginProprietaire dans la table etreInvite avec le sondage et avec le booléen est admin à true.

CodeSecretSondage est unique pour chaque sondage.

CRENEAU(dateEtHoraire, idSondage#)

ETREINVITE(loginInvite#, idSondage#, estAdmin, presenceRequise)

ETREDISPONIBLE(loginUtilisateur#, dateEtHoraire#, idSondage#, typeDisponibilite#)

Le couple loginUtilisateur, idSondage pointe vers la table etreInvite et le couple dateEtHoraire, idSondage pointe vers Créneau.

DISPONIBILITE(typeDisponibilite)

typeDisponibilite est dans sa propre table afin de permettre l'intégration de nouveaux types de disponibilité.

